JWT Authentication on Frontend Using Fetch API

1. Overview of JWT Authentication Flow

1.1 Basic Server-Side Flow

Before implementing the frontend logic:

1. User submits username and password.
2. Server (Node.js + Express) validates credentials against the database.
3. If valid, server returns:
    - A **JWT token**
    - Basic information (username, email, role, etc.)
4. The client receives the JWT token and must **store it** (localStorage or cookies).
5. For all **protected API requests**, the client must send the token via the header:

```
Authorization: Bearer <token>
```

2. Using localStorage for JWT

2.1 Methods Used

Method	Description
`localStorage.setItem(key, value)`	Store value
`localStorage.getItem(key)`	Retrieve value
`localStorage.removeItem(key)`	Remove a specific key
`localStorage.clear()`	Remove all keys

In this project:

- Save token after login → `setItem('token', token)`
- Read token on protected pages → `getItem('token')`
- Remove token on logout → `removeItem('token')`

3. Using Fetch API for Requests

3.1 Basic Fetch Syntax

```
fetch(url, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer <token>"   // for protected APIs
  },
  body: JSON.stringify(data)
});
```

Content Types:

- JSON: `application/json`
- Form Data: default `application/x-www-form-urlencoded` (not used here)

---

4. Project Structure (Frontend)

```
project/
  ├── index.twig (Login Page)
  ├── register.twig (Register Page)
  ├── students.twig (Protected Page)
  ├── js/
  │     └── scripts.js   (Main frontend JS)
```

---

5. Register Page Implementation

5.1 HTML Form (Twig Example)

```
<form id="registerForm">
    <input id="registerUser" type="text" placeholder="Username">
    <input id="registerEmail" type="email" placeholder="Email">
    <input id="registerPassword" type="password" placeholder="Password">
    <button type="submit">Register</button>
</form>

<script src="js/scripts.js"></script>
```

5.2 JavaScript Logic for Registration

```
if (registerForm) {
    registerForm.addEventListener("submit", async (e) => {
        e.preventDefault();

        const username = document.getElementById("registerUser").value;
        const email = document.getElementById("registerEmail").value;
        const password = document.getElementById("registerPassword").value;

        const response = await fetch(`${API_URL}/register`, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ username, email, password })
        });

        const data = await response.json();

        if (response.ok) {
            alert("Registration successful. You can now log in.");
            window.location.href = "index.twig";
        } else {
            alert(data.message || "Registration failed.");
        }
    });
}
```

6. Login Page Implementation

6.1 HTML Form (Twig)

```
<form id="loginForm">
    <input id="loginUser" type="text" placeholder="Username">
    <input id="loginPassword" type="password" placeholder="Password">
    <button type="submit">Login</button>
</form>

<script src="js/scripts.js"></script>
```

6.2 JavaScript Logic for Login

```
if (loginForm) {
    loginForm.addEventListener("submit", async (e) => {
        e.preventDefault();

        const username = document.getElementById("loginUser").value;
        const password = document.getElementById("loginPassword").value;
```

```
        const response = await fetch(`${API_URL}/login`, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ username, password })
        });

        const data = await response.json();

        if (response.ok) {
            localStorage.setItem("token", data.token);
            window.location.href = "students.twig";
        } else {
            alert(data.message || "Login failed.");
        }
    });
}
```

7. Protecting Pages Using JWT

7.1 Token Check Function

```
function checkAuth() {
    const token = localStorage.getItem("token");

    if (!token) {
        window.location.href = "index.twig";
        return null;
    }

    return token;
}
```

7.2 Auto-run on Protected Pages

```
const token = checkAuth();
```

8. Fetching Protected Student Data

```
async function viewAllStudents() {
    const token = checkAuth();
```

```
    const response = await fetch(`${API_URL}/students`, {
        method: "GET",
        headers: {
            "Authorization": `Bearer ${token}`
        }
    });

    const data = await response.json();
    // render data using Twig or JS
}
```

---

9. Add, Update, Delete Student with Authorization Header

For **all CRUD operations**, include:

```
headers: {
    "Content-Type": "application/json",
    "Authorization": `Bearer ${token}`
}
```

Example: Delete Student

```
await fetch(`${API_URL}/students/${id}`, {
    method: "DELETE",
    headers: {
        "Authorization": `Bearer ${token}`
    }
});
```

---

10. Logout Functionality

10.1 Logout Button (Twig)

```
<button class="btn btn-danger" onclick="logout()">Logout</button>
```

10.2 Logout Logic

```
function logout() {
    localStorage.removeItem("token");
    window.location.href = "index.twig";
}
```

---

11. Viewing Stored Token in Browser DevTools

Steps:

1. Open Browser → Press F12
2. Go to: Application → Storage → Local Storage
3. View key `token`
4. Value will look like:

```
<ENCRYPTED_JWT_STRING>
```

The JWT has:

- Header
- Payload
- Signature

But payload is **not readable directly** because server returns it encoded.

---

12. Optional: Using Cookies Instead of localStorage

- Cookies are **more secure** (HttpOnly + Secure flags)
- They prevent token theft via JavaScript
- But require proper backend configuration

Final Summary

The tutorial demonstrates the complete implementation of:

- Registration
- Login
- Saving JWT token in localStorage
- Using Fetch API to send token in Authorization header
- Protecting routes/pages on the frontend
- Performing CRUD operations on protected Student APIs
- Logout and automatic redirect

- Viewing token in DevTools
